

TP3 — PySpark et MySQL : fédérer deux bases de données

Big Data Engineering — Master 1 Data Science — DMI/FST/UCAD — Prof. S. Ndiaye

Au TP2, vos données vivaient dans des fichiers CSV. Dans la vraie vie, elles vivent surtout dans des **SGBD opérationnels**. Notre plateforme e-commerce a grandi : le **service client** gère sa base MySQL (`ecommerce_crm`), l'équipe de la **plateforme de vente** gère la sienne (`ecommerce_ventes`). Deux bases, deux équipes... et vous, l'ingénieur data, devez croiser les deux.

Objectifs du TP :

1. lire des tables MySQL en **DataFrames** PySpark via **JDBC** ;
2. exposer ces DataFrames comme **vues temporaires** ;
3. **joindre** en Spark SQL des données issues de **deux bases différentes** — puis d'un CSV en prime : c'est la **fédération de sources**, la vraie puissance de Spark.

Consignes : exécutez les cellules **dans l'ordre**, complétez les blocs **À COMPLÉTER** , répondez aux questions en markdown, puis poussez le notebook **exécuté (sorties visibles)** sur votre dépôt GitHub avant la prochaine séance.

1. Vérification et réparation de l'environnement

⚠ **Exécutez ces cellules dans l'ordre, sans en sauter aucune.** Comme au TP2, elles diagnostiquent les pannes classiques et **réparent automatiquement** la plus fréquente. Environnement cible du cours : **environnement virtuel Python 3.11, Java 17, PySpark 3.5.1** — et pour ce TP, **MySQL 8** (déjà installé, avec Workbench).

Symptôme	Cause	Remède
ModuleNotFoundError: No module named 'pyspark'	le noyau Jupyter n'est pas celui de votre venv	la cellule 1.2 installe PySpark dans le noyau actif
Java gateway process exited / JAVA_HOME is not set	Java absent ou mal configuré	installer Java 17 (voir TP d'installation)
Python worker failed to connect back (Windows)	Spark ne trouve pas le bon interpréteur	la cellule 4.1 règle <code>PYSPARK_PYTHON</code> automatiquement
Hadoop bin directory does not exist: ... (Windows)	<code>HADOOP_HOME</code> mal pointé ou <code>winutils.exe</code> absent	la cellule 4.1 corrige ou bascule en plan B toute seule

Symptôme	Cause	Remède
Communications link failure	le serveur MySQL n'est pas démarré	la cellule 2.2 le détecte avant Spark, avec la marche à suivre

1.1 Quel Python utilise ce noyau ?

```
In [1]: import sys

print("Interpreteur Python utilise par CE noyau :")
print("    ", sys.executable)
print("Version :", sys.version.split()[0])

# Attendu : Le Python de votre environnement virtuel du cours (3.11.x).
# Si Le chemin ci-dessus ne pointe PAS vers votre venv, changez de noyau
# (menu Kernel > Change kernel). PAS DE PANIQUE si vous vous etes trompes :
# la cellule suivante repare ce cas automatiquement.
```

Interpreteur Python utilise par CE noyau :
 c:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\venv-bigdata\Scripts\python.exe
 Version : 3.13.13

1.2 PySpark est-il visible ? (installation automatique si besoin)

Cette cellule installe `pyspark==3.5.1` dans l'environnement exact du noyau actif (`sys.executable -m pip`) : elle fonctionne donc même si vous n'êtes pas sur le bon noyau ou si l'installation initiale a été faite hors du venv.

```
In [2]: import sys, subprocess

try:
    import pyspark
    print("OK : pyspark", pyspark.__version__, "est deja visible par ce noyau.")
except ModuleNotFoundError:
    print("pyspark est ABSENT de ce noyau -> installation en cours (1-2 min)...")
    subprocess.check_call([sys.executable, "-m", "pip", "install",
                           "pyspark==3.5.1"])

    import importlib
    importlib.invalidate_caches()
    import pyspark
    print("Installation terminee : pyspark", pyspark.__version__)
```

OK : pyspark 4.2.0 est deja visible par ce noyau.

1.3 Java est-il là ?

Spark tourne sur la **JVM** : sans Java, pas de SparkSession. Le cours cible **Java 17**.

```
In [3]: import subprocess

try:
    resultat = subprocess.run(["java", "-version"],
```

```

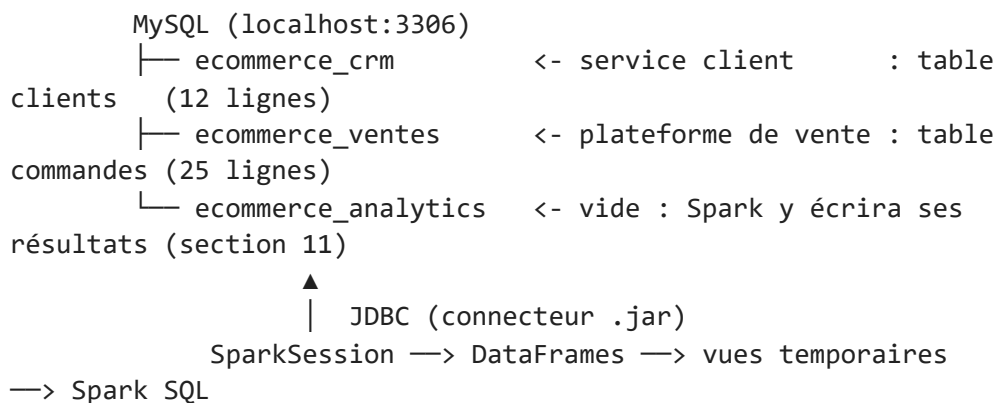
capture_output=True, text=True)
ligne = (resultat.stderr or resultat.stdout).splitlines()[0]
print("Java detecte :", ligne)
print("Attendu : version 17.x (ex. openjdk version \"17.0. ...\")")
except FileNotFoundError:
    print("PROBLEME : la commande 'java' est introuvable.")
    print("-> Installez Java 17 (Temurin/OpenJDK) puis relancez ce noyau.")
    print("-> Windows : vérifiez aussi la variable d'environnement JAVA_HOME.")

```

Java detecte : java version "17.0.11" 2024-04-16 LTS

Attendu : version 17.x (ex. openjdk version "17.0. ...")

2. Nos deux bases MySQL



💡 **Point d'architecture.** La table `commandes` référence `client_id` ... qui vit dans **une autre base, gérée par une autre équipe**. Aucune clé étrangère ne relie les deux : l'intégrité n'est **pas garantie** par MySQL. C'est un cas réel très courant — et c'est Spark qui fera la police (section 8).

2.1 Avant toute chose : le script `init_tp3.sql`

Si ce n'est pas déjà fait, exécutez **une fois** le script fourni `init_tp3.sql`, **en tant que root** :

- **MySQL Workbench** (recommandé) : connexion root → *File* ▶ *Open SQL Script...* → `init_tp3.sql` → cliquer sur l'éclair ⚡. Les 4 requêtes de vérification finales doivent afficher **12**, **25**, `spark_user` et **3** ;
- **ou en ligne de commande** : `mysql -u root -p < init_tp3.sql`.

Le script crée les 3 bases **et** l'utilisateur `spark_user` (lecture seule sur les bases métier) : le notebook n'a ainsi jamais besoin du mot de passe root.

2.2 Le serveur MySQL répond-il ?

On le vérifie **avant** de lancer Spark : un simple test TCP sur le port 3306 donne un diagnostic immédiat et lisible.

```
In [4]: import socket

MYSQL_HOTE = "127.0.0.1"
MYSQL_PORT = 3306

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(3)
if s.connect_ex((MYSQL_HOTE, MYSQL_PORT)) == 0:
    print(f"OK : un serveur repond sur {MYSQL_HOTE}:{MYSQL_PORT} (MySQL démarre)")
else:
    print(f"PROBLEME : rien ne repond sur {MYSQL_HOTE}:{MYSQL_PORT}.")
    print("-> Démarrez le serveur MySQL puis re-exécutez cette cellule :")
    print("  Windows : services.msc > service 'MySQL80' > Démarrer")
    print("  macOS   : Preferences Systeme > MySQL > Start (ou brew services s")
    print("  Linux   : sudo systemctl start mysql")
s.close()
```

OK : un serveur repond sur 127.0.0.1:3306 (MySQL démarre).

2.3 Les paramètres de connexion

Tout est regroupé ici : si votre configuration diffère (port non standard, autre mot de passe...), **c'est la seule cellule à modifier**.

Les deux options de l'URL JDBC évitent deux pièges classiques de MySQL 8 :

`allowPublicKeyRetrieval=true` (erreur « *Public Key Retrieval is not allowed* ») et
`useSSL=false` (avertissements SSL sur un serveur local).

```
In [5]: MYSQL_UTILISATEUR = "spark_user"      # cree par init_tp3.sql
        MYSQL_MDP         = "Ucad2026!"      # idem (droits SELECT seulement)

OPTIONS_JDBC = "?useSSL=false&allowPublicKeyRetrieval=true"

URL_CRM      = f"jdbc:mysql://{MYSQL_HOTE}:{MYSQL_PORT}/ecommerce_crm{OPTIONS_JDBC}"
URL_VENTES   = f"jdbc:mysql://{MYSQL_HOTE}:{MYSQL_PORT}/ecommerce_ventes{OPTIONS_JDBC}"
URL_ANALYTICS = f"jdbc:mysql://{MYSQL_HOTE}:{MYSQL_PORT}/ecommerce_analytics{OPTIONS_JDBC}"

RELEVES = {} # notre tableau de relevés, rempli au fil du TP (section 12)

print("URL de la base CRM      :", URL_CRM)
print("URL de la base ventes   :", URL_VENTES)
```

URL de la base CRM : jdbc:mysql://127.0.0.1:3306/ecommerce_crm?useSSL=false&allowPublicKeyRetrieval=true
 URL de la base ventes : jdbc:mysql://127.0.0.1:3306/ecommerce_ventes?useSSL=false&allowPublicKeyRetrieval=true

2.4 Plan B : (re)créer les bases depuis le notebook (optionnel)

Si Workbench n'est pas disponible sur votre poste, passez `EXECUTER_PLAN_B` à `True` : la cellule installe le petit pilote `mysql-connector-python`, vous demande le mot de passe **root**, puis exécute `init_tp3.sql` pour vous. Sinon, laissez `False` et passez à la suite.

```
In [6]: EXECUTER_PLAN_B = False # passez a True uniquement si besoin

if EXECUTER_PLAN_B:
    import sys, subprocess, getpass
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
                           "mysql-connector-python"])
    import mysql.connector
    mdp_root = getpass.getpass("Mot de passe root MySQL : ")
    connexion = mysql.connector.connect(host=MYSQL_HOTE, port=MYSQL_PORT,
                                       user="root", password=mdp_root)

    curseur = connexion.cursor()
    with open("init_tp3.sql", encoding="utf-8") as f:
        for resultat in curseur.execute(f.read(), multi=True):
            if resultat.with_rows:
                print(resultat.statement.splitlines()[0], "->",
                      resultat.fetchall())
    connexion.commit(); curseur.close(); connexion.close()
    print("Bases (re)creées : passez a la section 3.")
else:
    print("Plan B desactive : les bases ont ete creees via Workbench (2.1).")
```

Plan B desactive : les bases ont ete creees via Workbench (2.1).

3. Le connecteur JDBC : le pont entre Spark et MySQL

💡 **Pourquoi un fichier .jar alors qu'on code en Python ?** PySpark n'est qu'une façade : le moteur Spark tourne sur la **JVM**, et c'est elle qui parle aux bases de données via **JDBC** (*Java DataBase Connectivity*), le standard Java d'accès aux SGBD. Chaque SGBD fournit son **connecteur** : un **.jar** qui traduit JDBC vers son protocole réseau. Pour MySQL, c'est **MySQL Connector/J**.

La cellule ci-dessous :

1. cherche un connecteur déjà présent **à côté du notebook** (MySQL Connector/J, ou son cousin MariaDB Connector/J — protocole compatible : MariaDB est un *fork* de MySQL) ;
2. sinon, **télécharge** le connecteur officiel depuis Maven Central (le « PyPI de l'écosystème Java ») ;
3. en déduit la **classe du pilote** que Spark devra charger.

```
In [7]: import glob, urllib.request
from pathlib import Path

VERSION_CONNECTEUR = "8.4.0"
URL_CONNECTEUR = ("https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/"
                  + VERSION_CONNECTEUR
                  + "/mysql-connector-j-" + VERSION_CONNECTEUR + ".jar")

# Un vrai connecteur pese > 500 Ko : un fichier minuscule est un debris
# de telechargement interrompu, a ignorer (et a signaler)
```

```

TAILLE_MINI = 500_000
connecteurs = []
for jar in (sorted(glob.glob("mysql-connector-j-*.jar"))
            + sorted(glob.glob("mariadb-java-client*.jar"))):
    if Path(jar).stat().st_size >= TAILLE_MINI:
        connecteurs.append(jar)
    else:
        print("ATTENTION :", jar, "est trop petit (telechargement",
              "interrompu ?) -> ignore. Supprimez-le.")

if connecteurs:
    JAR_JDBC = connecteurs[0]
    print("Connecteur deja present :", JAR_JDBC)
else:
    print("Aucun connecteur local : telechargement depuis Maven Central...")
    try:
        JAR_JDBC = "mysql-connector-j-" + VERSION_CONNECTEUR + ".jar"
        urllib.request.urlretrieve(URL_CONNECTEUR, JAR_JDBC)
        print("Telechargement OK :", JAR_JDBC)
    except Exception as erreur:
        raise RuntimeError(
            "Echec du telechargement (" + str(erreur) + ").\n"
            "-> Telechargez le .jar manuellement puis placez-le A COTE du "
            "notebook :\n    " + URL_CONNECTEUR + "\n"
            "    (ou dev.mysql.com > Connector/J > Platform Independent, "
            "le .jar est dans le ZIP)") from None

# La classe du pilote depend du connecteur trouve
if "mysql-connector" in JAR_JDBC:
    PILOTE_JDBC = "com.mysql.cj.jdbc.Driver"
else:
    PILOTE_JDBC = "org.mariadb.jdbc.Driver"

JAR_JDBC = str(Path(JAR_JDBC).resolve())
print("Classe du pilote      :", PILOTE_JDBC)
print("Chemin absolu du .jar  :", JAR_JDBC)

```

```

Connecteur deja present : mysql-connector-j-8.4.0.jar
Classe du pilote      : com.mysql.cj.jdbc.Driver
Chemin absolu du .jar  : C:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\notebooks\mysql-connector-j-8.4.0.jar

```

4. La SparkSession, branchée sur MySQL

4.1 Verrouiller Python et dompter Windows (cellule anti-pièges)

Deux protections en une :

1. comme au TP2, on impose à Spark le Python **de ce noyau** (piège Python worker failed to connect back) ;
2. **nouveau piège du TP3, spécifique à Windows** : charger un .jar par spark.jars déclenche un mécanisme Hadoop qui exige winutils.exe .

Symptôme : `Hadoop bin directory does not exist: ...` au démarrage de la session. La cellule diagnostique votre poste et choisit toute seule la bonne stratégie :

Situation détectée	Réaction de la cellule
Linux / macOS	rien à faire, <code>spark.jars</code> fonctionne
Windows, <code>HADOOP_HOME</code> pointe sur <code>...\bin</code>	corrige la variable pour la session (piège : elle doit pointer sur le dossier <i>parent</i> de <code>bin</code>)
Windows, <code>winutils.exe</code> introuvable	bascule sur le plan B : chargement du connecteur par <code>extraClassPath</code> , qui n'a pas besoin de Hadoop

```
In [8]: import os

JAR_JDBC = os.path.join(
    os.getcwd(),
    "mysql-connector-j-8.4.0.jar"
)

print("JAR JDBC :", JAR_JDBC)
print("Existe :", os.path.exists(JAR_JDBC))
```

```
JAR JDBC : c:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\notebooks\mysql-connector-j-8.4.0.jar
Existe : True
```

```
In [9]: import os, sys
from pathlib import Path

os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
print("Interpreteur impose a Spark :", sys.executable)

# Strategie de chargement du connecteur (ajustee ci-dessous si Windows)
UTILISER_SPARK_JARS = True

if os.name == "nt":
    hadoop = os.environ.get("HADOOP_HOME", "").strip(' ')

    # Piegé n. 1 : HADOOP_HOME pointe sur ...\\bin au lieu du dossier parent
    if (hadoop.rstrip("\\").lower().endswith("bin")
        and (Path(hadoop) / "winutils.exe").exists()):
        os.environ["HADOOP_HOME"] = str(Path(hadoop).parent)
        hadoop = os.environ["HADOOP_HOME"]
        print("HADOOP_HOME corrige automatiquement ->", hadoop)
        print("(pensez a corriger la variable Windows de facon permanente)")

    winutils = Path(hadoop) / "bin" / "winutils.exe" if hadoop else None
    if hadoop and winutils.exists():
        os.environ["PATH"] = (str(Path(hadoop) / "bin") + os.pathsep
                               + os.environ["PATH"])
        print("winutils.exe detecte :", winutils, "-> spark.jars utilisable")
    else:
        UTILISER_SPARK_JARS = False
        print("winutils.exe introuvable -> PLAN B automatique :")
        print("le connecteur sera charge par extraClassPath (sans Hadoop).")
else:
    print("Systeme non-Windows : aucun reglage Hadoop necessaire.")
```

Interpreteur impose a Spark : c:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\venv-bigdata\Scripts\python.exe
 winutils.exe detecte : C:\hadoop\bin\winutils.exe -> spark.jars utilisable

4.2 Créer la session avec le connecteur

La **seule nouveauté** par rapport au TP2 est l'embarquement du connecteur dans la JVM de Spark : par `spark.jars` (la voie canonique, celle des clusters), ou par `extraClassPath` si la cellule 4.1 a choisi le plan B Windows. ⚠ Dans les deux cas, cela se fixe **à la création** de la session — si vous changez de `.jar` ou corrigez votre configuration, **redémarrez le noyau** (Kernel ▶ Restart & Run All).

```
In [10]: from pyspark.sql import SparkSession
from pyspark.sql import functions as F

constructeur = (SparkSession.builder
    .appName("TP3 - PySpark et MySQL - UCAD")
    .master("local[*]")
    .config("spark.driver.host", "127.0.0.1")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.ui.showConsoleProgress", "false"))

if UTILISER_SPARK_JARS:
    constructeur = constructeur.config("spark.jars", JAR_JDBC)
    print("Connecteur charge via spark.jars")
else:
    constructeur = (constructeur
        .config("spark.driver.extraClassPath", JAR_JDBC)
        .config("spark.executor.extraClassPath", JAR_JDBC))
    print("Connecteur charge via extraClassPath (plan B Windows)")

spark = constructeur.getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

print("SparkSession creee !")
print("Version de Spark :", spark.version)
print("Interface web : http://localhost:4040 (onglet SQL / JDBC)")
```

Connecteur charge via spark.jars

```
c:\Users\24kha\Desktop\MSIA_ISI\TP BIG DATA\venv-bigdata\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use pandas < 3.0.0 for now.
```

```
require_minimum_pandas_version()
```

SparkSession creee !

Version de Spark : 4.2.0

Interface web : http://localhost:4040 (onglet SQL / JDBC)

5. Lire une table MySQL en DataFrame



Ce qu'il faut comprendre

La lecture JDBC suit toujours le même gabarit :

```
spark.read.format("jdbc")
    .option("url", ...)          # quelle base
    .option("dbtable", ...)      # quelle table (ou requête, section 10)
    .option("user", ...) / .option("password", ...)
    .option("driver", ...)       # la classe du pilote (section 3)
    .load()
```

Deux différences majeures avec la lecture CSV du TP2 :

	CSV (TP2)	MySQL via JDBC (TP3)
Schéma	deviné (<code>inferSchema</code> , 2 lectures) ou écrit à la main	fourni par le SGBD : fiable, aucune option
Types	tout risque d'arriver en <code>string</code>	<code>DATE</code> → <code>date</code> , <code>INT</code> → <code>int</code> ... garantis

5.1 Premier essai : la table `clients` de la base CRM

```
In [11]: import traceback

try:
    df_clients = (spark.read.format("jdbc")
        .option("url", URL_CRM)
        .option("dbtable", "clients")
        .option("user", MYSQL_UTILISATEUR)
        .option("password", MYSQL_MDP)
        .option("driver", PILOTE_JDBC)
        .load())

    df_clients.show()
    print("SUCCES")
except Exception as e:
    with open("erreur_jdbc.txt", "w", encoding="utf-8") as f:
        traceback.print_exc(file=f)
    print("Erreur capturee dans erreur_jdbc.txt")
    print()
    print(str(e)[:3000])
```

```

+-----+-----+-----+-----+-----+-----+
-----+
|client_id|      nom|      ville|  telephone|      email|date_insc
ription|
+-----+-----+-----+-----+-----+-----+
-----+
|      C001|    Awa Diop|    Dakar|77 123 45 67|  awa.diop@gmail.com|      20
24-09-12|
|      C002| Moussa Ndiaye|    Thies|78 234 56 78|moussa.ndiaye@yah...|      20
24-10-03|
|      C003|    Fatou Sall|    Dakar|76 345 67 89|fatou.sall@gmail.com|      20
24-11-21|
|      C004|    Cheikh Ba|    Dakar|70 456 78 90|      NULL|      20
25-01-15|
|      C005|Aissatou Diallo|Saint-Louis|75 567 89 01|aissatou.diallo@h...|      20
25-02-08|
|      C006|  Ibrahima Fall|    Kaolack|77 678 90 12|ibrahima.fall@gma...|      20
25-03-19|
|      C007|    Ousmane Sarr| Ziguinchor|78 789 01 23|ousmane.sarr@gmai...|      20
25-04-27|
|      C008|    Khady Gueye|    Dakar|76 890 12 34|khady.gueye@yahoo.fr|      20
25-05-30|
|      C009|    Mamadou Sy|    Touba|70 901 23 45|      NULL|      20
25-06-14|
|      C010|    Adama Kane|    Rufisque|77 012 34 56|adama.kane@gmail.com|      20
25-07-22|
|      C011|    Bineta Mbaye|    Mbour|75 123 45 67|bineta.mbaye@gmai...|      20
25-08-09|
|      C012|  Serigne Diouf|    Louga|78 210 43 65|serigne.diouf@hot...|      20
25-09-05|
+-----+-----+-----+-----+-----+-----+
-----+

```

SUCCES

```

In [12]: df_clients = (spark.read.format("jdbc")
    .option("url", URL_CRM)
    .option("dbtable", "clients")
    .option("user", MYSQL_UTILISATEUR)
    .option("password", MYSQL_MDP)
    .option("driver", PILOTE_JDBC)
    .load())

df_clients.show()

# Observation attendue : 12 clients, villes du Senegal, 2 emails NULL

```

```

+-----+-----+-----+-----+-----+-----+
-----+
|client_id|          nom|      ville|  telephone|          email|date_inscription|
+-----+-----+-----+-----+-----+-----+
-----+
|      C001|      Awa Diop|      Dakar|77 123 45 67|  awa.diop@gmail.com|      20
24-09-12|
|      C002| Moussa Ndiaye|      Thies|78 234 56 78|moussa.ndiaye@yah...|      20
24-10-03|
|      C003|      Fatou Sall|      Dakar|76 345 67 89|fatou.sall@gmail.com|      20
24-11-21|
|      C004|      Cheikh Ba|      Dakar|70 456 78 90|          NULL|      20
25-01-15|
|      C005|Aissatou Diallo|Saint-Louis|75 567 89 01|aissatou.diallo@h...|      20
25-02-08|
|      C006|  Ibrahima Fall|      Kaolack|77 678 90 12|ibrahima.fall@gma...|      20
25-03-19|
|      C007|      Ousmane Sarr| Ziguinchor|78 789 01 23|ousmane.sarr@gmai...|      20
25-04-27|
|      C008|      Khady Gueye|      Dakar|76 890 12 34|khady.gueye@yahoo.fr|      20
25-05-30|
|      C009|      Mamadou Sy|      Touba|70 901 23 45|          NULL|      20
25-06-14|
|      C010|      Adama Kane|      Rufisque|77 012 34 56|adama.kane@gmail.com|      20
25-07-22|
|      C011|      Bineta Mbaye|      Mbour|75 123 45 67|bineta.mbaye@gmai...|      20
25-08-09|
|      C012|      Serigne Diouf|      Louga|78 210 43 65|serigne.diouf@hotmail...|      20
25-09-05|
+-----+-----+-----+-----+-----+-----+
-----+

```

```
In [13]: df_clients.printSchema()
```

```

# Observation attendue : Les types viennent DIRECTEMENT de MySQL --
# date_inscription est deja un vrai type date, sans inferSchema ni cast !

```

```

root
|-- client_id: string (nullable = true)
|-- nom: string (nullable = true)
|-- ville: string (nullable = true)
|-- telephone: string (nullable = true)
|-- email: string (nullable = true)
|-- date_inscription: date (nullable = true)

```

```
In [14]: RELEVES["nb_clients (base CRM)"] = df_clients.count()
print("Nombre de clients :", RELEVES["nb_clients (base CRM)"])
```

Nombre de clients : 12

5.2 Industrialisons : une petite fonction de lecture

Trois tables à lire, un seul gabarit : on factorise (réflexe d'ingénieur du TP2 !).

```
In [15]: def lire_table_mysql(url_base, table):
        """Lit une table MySQL et renvoie un DataFrame Spark."""
```

```

return (spark.read.format("jdbc")
        .option("url", url_base)
        .option("dbtable", table)
        .option("user", MYSQL_UTILISATEUR)
        .option("password", MYSQL_MDP)
        .option("driver", PILOTE_JDBC)
        .load())

print("Fonction definie : lire_table_mysql(url_base, table)")

```

Fonction definie : lire_table_mysql(url_base, table)

À vous de jouer 5.a

Avec `lire_table_mysql`, chargez la table `commandes` de la **seconde base** (`URL_VENTES`) dans un DataFrame `df_commandes`, puis :

1. affichez ses 5 premières lignes ;
2. affichez son schéma ;
3. comptez ses lignes et rangez le résultat dans `RELEVES["nb_commandes (base ventes)"]`.

```

In [16]: # === À COMPLÉTER ===
df_commandes = lire_table_mysql(URL_VENTES, "commandes")

# 1. Les 5 premières lignes
df_commandes.show(5)

# 2. Le schema
df_commandes.printSchema()

# 3. Le comptage, range dans RELEVES
RELEVES["nb_commandes (base ventes)"] = df_commandes.count()
print("Nombre de commandes :", RELEVES["nb_commandes (base ventes)"])

```

```

+-----+-----+-----+-----+-----+-----+-----+
--+
|commande_id|client_id|produit_id|montant_fcfa|moyen_paiement|statut|date_commande|
+-----+-----+-----+-----+-----+-----+-----+
--+
|      CMD01|      C001|      P001|      145000|  Orange Money|livree|  2025-10-02|
|      CMD02|      C003|      P002|      25000|           Wave|livree|  2025-10-05|
|      CMD03|      C002|      P010|      8500|    especes|livree|  2025-10-07|
|      CMD04|      C001|      P003|      32000|  Orange Money|livree|  2025-10-12|
|      CMD05|      C008|      P005|      57500|         carte|livree|  2025-10-15|
+-----+-----+-----+-----+-----+-----+-----+
--+
only showing top 5 rows
root
|-- commande_id: string (nullable = true)
|-- client_id: string (nullable = true)
|-- produit_id: string (nullable = true)
|-- montant_fcfa: integer (nullable = true)
|-- moyen_paiement: string (nullable = true)
|-- statut: string (nullable = true)
|-- date_commande: date (nullable = true)

```

Nombre de commandes : 25

 **Faites le point.** Vous avez maintenant, dans la même SparkSession, deux DataFrames issus de **deux bases de données différentes**. Ils sont désormais des citoyens Spark ordinaires : tout ce que vous avez appris au TP2 (transformations, agrégations, jointures...) s'applique tel quel.

6. Les vues temporaires : parler SQL à ses DataFrames

Ce qu'il faut comprendre

`createOrReplaceTempView("nom")` enregistre un DataFrame sous un **nom de table** dans le catalogue de la session. On peut alors l'interroger en SQL standard avec `spark.sql(...)` — qui renvoie... un DataFrame, bien sûr.

- **temporaire** : la vue disparaît avec la session (rien n'est écrit dans MySQL) ;
- `OrReplace` : ré-exécuter la cellule ne provoque pas d'erreur ;
- la requête SQL est optimisée par le **même moteur** (Catalyst) que l'API DataFrame : c'est un choix de style, pas de performance.

6.1 Enregistrer nos deux vues

```
In [17]: df_clients.createOrReplaceTempView("clients")
df_commandes.createOrReplaceTempView("commandes")

print("Vues enregistrées dans le catalogue de la session :")
for table in spark.catalog.listTables():
    print(" -", table.name, "(temporaire :", str(table.isTemporary) + ")")
```

Vues enregistrées dans le catalogue de la session :

- clients (temporaire : True)
- commandes (temporaire : True)

6.2 Premières requêtes SQL — une vue à la fois

```
In [18]: # Sur la vue clients (donnees de la base CRM)
spark.sql("""
    SELECT ville, COUNT(*) AS nb_clients
    FROM clients
    GROUP BY ville
    ORDER BY nb_clients DESC, ville
""").show()

# Observation attendue : Dakar en tete avec 4 clients, 9 villes en tout
```

ville	nb_clients
Dakar	4
Kaolack	1
Louga	1
Mbour	1
Rufisque	1
Saint-Louis	1
Thies	1
Touba	1
Ziguinchor	1

```
In [19]: # Sur La vue commandes (donnees de La base ventes)
spark.sql("""
    SELECT statut,
           COUNT(*)           AS nb,
           SUM(montant_fcfa) AS total_fcfa
    FROM commandes
    GROUP BY statut
    ORDER BY nb DESC
""").show()

# Observation attendue : 19 livrees, 3 en_cours, 3 annulees
```

statut	nb	total_fcfa
livree	19	997500
en_cours	3	76000
annulee	3	66500

🏆 À vous de jouer 6.a

En **une requête Spark SQL** sur la vue `commandes`, affichez le **top 5 des commandes par montant** : colonnes `commande_id`, `montant_fcfa`, `moyen_paiement`, triées par montant décroissant, limitées à 5 lignes.

Indice : `ORDER BY ... DESC` puis `LIMIT 5`.

```
In [20]: spark.sql("""
    SELECT commande_id, montant_fcfa, moyen_paiement
    FROM commandes
    ORDER BY montant_fcfa DESC
    LIMIT 5
""").show()
```

commande_id	montant_fcfa	moyen_paiement
CMD01	145000	Orange Money
CMD16	145000	Orange Money
CMD19	145000	carte
CMD12	74500	carte
CMD07	74500	Orange Money

7. 🎯 La jointure inter-bases : le moment clé du TP

Les vues `clients` et `commandes` viennent de **deux bases MySQL différentes**. Pour Spark, peu importe : ce sont deux tables de son catalogue. La jointure s'écrit donc... comme n'importe quelle jointure.

7.1 En Spark SQL

```
In [21]: df_jointure = spark.sql("""
        SELECT co.commande_id,
               c.nom,
               c.ville,
               co.montant_fcfa,
               co.moyen_paiement,
               co.statut
        FROM clients AS c
        JOIN commandes AS co
        ON c.client_id = co.client_id
        """)

df_jointure.show(8)
print("Lignes dans la jointure :", df_jointure.count())

# Observation attendue : 24 lignes... alors qu'il y a 25 commandes.
# Ou est passée la 25e ? Réponse en section 8 !
```

commande_id	nom	ville	montant_fcfa	moyen_paiement	statut
CMD01	Awa Diop	Dakar	145000	Orange Money	livree
CMD04	Awa Diop	Dakar	32000	Orange Money	livree
CMD13	Awa Diop	Dakar	25000	Wave	livree
CMD24	Awa Diop	Dakar	74500	carte	livree
CMD03	Moussa Ndiaye	Thies	8500	especes	livree
CMD16	Moussa Ndiaye	Thies	145000	Orange Money	livree
CMD02	Fatou Sall	Dakar	25000	Wave	livree
CMD10	Fatou Sall	Dakar	28500	Orange Money	livree

only showing top 8 rows
Lignes dans la jointure : 24

💡 **Ce qui vient de se passer, en une phrase :** une requête SQL a croisé des données de `ecommerce_crm` et `ecommerce_ventes` **sans qu'aucune des deux bases ne voie l'autre** — Spark a tiré les deux tables vers lui et fait la jointure dans son moteur distribué.

Honnêteté intellectuelle : deux bases d'un *même* serveur MySQL peuvent se joindre en SQL pur (`ecommerce_crm.clients JOIN ecommerce_ventes.commandes`). La vraie puissance de Spark est ailleurs :

- les bases peuvent être sur des **serveurs différents**, voire des **SGBD différents** (MySQL + PostgreSQL + Oracle...);
- on peut y mêler des **fichiers** (CSV, JSON, Parquet...) — démonstration en section 9 ;
- le calcul est **distribuable** sur un cluster quand les volumes explosent ;
- le tout dans **un seul langage** et un seul plan optimisé.

7.2 La même jointure avec l'API DataFrame

Rappel du TP2 : SQL et API DataFrame sont deux syntaxes pour le même moteur.

```
In [22]: (df_clients
          .join(df_commandes, on="client_id", how="inner")
          .select("commande_id", "nom", "ville", "montant_fcfa", "statut")
          .show(5))

# Meme resultat que La version SQL : choisissez la syntaxe qui vous parle
```

commande_id	nom	ville	montant_fcfa	statut
CMD01	Awa Diop	Dakar	145000	livree
CMD04	Awa Diop	Dakar	32000	livree
CMD13	Awa Diop	Dakar	25000	livree
CMD24	Awa Diop	Dakar	74500	livree
CMD03	Moussa Ndiaye	Thies	8500	livree

only showing top 5 rows

7.3 Des questions métier, enfin !

Maintenant que les deux mondes sont réunis, les questions intéressantes deviennent possibles.

```
In [23]: # Chiffre d'affaires LIVRE par ville
df_ca_ville = spark.sql("""
    SELECT c.ville,
           COUNT(*)           AS nb_commandes,
           SUM(co.montant_fcfa) AS ca_fcfa
    FROM clients AS c
    JOIN commandes AS co ON c.client_id = co.client_id
    WHERE co.statut = 'livree'
    GROUP BY c.ville
    ORDER BY ca_fcfa DESC
""")
df_ca_ville.show()
```

*# Observation attendue : Dakar tres Largement en tete (~La moitie du CA).
 # Remarquez : Saint-Louis a DISPARU -- ses 2 commandes sont en_cours ou
 # annulee. Un GROUP BY ne montre que ce qui existe !*

ville	nb_commandes	ca_fcfa
Dakar	9	494000
Rufisque	2	160500
Thies	2	153500
Louga	2	86500
Touba	2	66000
Kaolack	1	8500

In [24]: *# Panier moyen par moyen de paiement (mobile money vs autres)*

```
spark.sql("""
    SELECT co.moyen_paiement,
           COUNT(*) AS nb,
           ROUND(AVG(co.montant_fcfa), 0) AS panier_moyen_fcfa
    FROM commandes AS co
    GROUP BY co.moyen_paiement
    ORDER BY panier_moyen_fcfa DESC
""").show()

# Observation attendue : 4 moyens de paiement ; Orange Money et Wave
# dominant en volume (le mobile money, signature du e-commerce senegalais)
```

moyen_paiement	nb	panier_moyen_fcfa
carte	4	87875.0
Orange Money	8	66250.0
Wave	8	22313.0
especes	5	16000.0

```
In [25]: RELEVES["ca_livre_total_fcfa"] = (
    df_ca_ville.agg(F.sum("ca_fcfa")).collect()[0][0])
RELEVES["ville_top_ca"] = df_ca_ville.first()["ville"]
print("CA livre total :", RELEVES["ca_livre_total_fcfa"], "FCFA")
print("Ville en tete :", RELEVES["ville_top_ca"])
```

CA livre total : 969000 FCFA

Ville en tete : Dakar

À vous de jouer 7.a

En Spark SQL, produisez le **palmarès des clients** : pour chaque client ayant au moins une commande **livrée**, affichez `nom`, `ville`, le nombre de commandes livrées (`nb_livrees`) et le total dépensé (`total_fcfa`), triés par `total_fcfa` décroissant.

Indice : même squelette que le CA par ville, en groupant par `c.nom`, `c.ville`.

```
In [26]: spark.sql("""
    SELECT
```

```

        c.nom,
        c.ville,
        COUNT(*) AS nb_livrees,
        SUM(cmd.montant_fcfa) AS total_fcfa
    FROM clients c
    JOIN commandes cmd ON c.client_id = cmd.client_id
    WHERE cmd.statut = 'livree'
    GROUP BY c.nom, c.ville
    ORDER BY total_fcfa DESC
    """).show()

```

nom	ville	nb_livrees	total_fcfa
Awa Diop	Dakar	4	276500
Adama Kane	Rufisque	2	160500
Moussa Ndiaye	Thies	2	153500
Cheikh Ba	Dakar	2	106500
Serigne Diouf	Louga	2	86500
Mamadou Sy	Touba	2	66000
Khady Gueye	Dakar	1	57500
Fatou Sall	Dakar	2	53500
Ibrahima Fall	Kaolack	1	8500

🤖 À vous de jouer 7.b

Même question en **API DataFrame** cette fois : à partir de `df_clients` et `df_commandes`, retrouvez le **CA livré par ville** (section 7.3), avec `filter`, `join`, `groupBy` / `agg` et `orderBy`.

Indice : `F.sum("montant_fcfa").alias("ca_fcfa")` dans le `agg`.

```

In [27]: # === À COMPLÉTER ===
(df_clients
    .join(df_commandes.filter(F.col("statut") == "livree"),
        df_clients.client_id == df_commandes.client_id)
    .groupBy("ville")
    .agg(F.sum("montant_fcfa").alias("ca_fcfa"))
    .orderBy(F.desc("ca_fcfa"))
    .show())

```

ville	ca_fcfa
Dakar	494000
Rufisque	160500
Thies	153500
Louga	86500
Touba	66000
Kaolack	8500

8. La police de l'intégrité : où est passée la 25e commande ?

Rappel de la section 7.1 : la jointure interne renvoie **24** lignes pour **25** commandes. Comme aucune clé étrangère ne relie les deux bases, rien n'a empêché la plateforme d'enregistrer une commande pour un client... inconnu du CRM. La jointure interne l'a **silencieusement éliminée** — le pire des scénarios : une perte de données invisible.

Les jointures de diagnostic (revoir TP2, section jointures)

Jointure	Question à laquelle elle répond
<code>left_anti</code>	quelles lignes de gauche n'ont pas de correspondance à droite ?
<code>left_semi</code>	quelles lignes de gauche ont une correspondance (sans dupliquer) ?

8.1 Les commandes orphelines(`left_anti`)

```
In [28]: df_orphelines = df_commandes.join(df_clients, on="client_id",
                                             how="left_anti")
df_orphelines.show()

RELEVES["nb_commandes_orphelines"] = df_orphelines.count()
print("Commandes orphelines :", RELEVES["nb_commandes_orphelines"])

# Observation attendue : 1 commande (CMD15), client C999 inconnu du CRM.
# En production : a signaler a l'equipe de la plateforme de vente !
```

```
+-----+-----+-----+-----+-----+-----+-----+
--
|client_id|commande_id|produit_id|montant_fcfa|moyen_paiement|statut|date_commande|
+-----+-----+-----+-----+-----+-----+-----+
--
|      C999|      CMD15|      P004|      28500|      Wave|livree|  2025-11-17|
+-----+-----+-----+-----+-----+-----+-----+
--
```

Commandes orphelines : 1

8.2 Et dans l'autre sens : les clients sans commande

Le `left_anti` inversé répond à une question **marketing** : qui est inscrit mais n'a jamais commandé ? (cible idéale d'une relance...)

```
In [29]: df_sans_commande = df_clients.join(df_commandes, on="client_id",
                                              how="left_anti")
df_sans_commande.select("client_id", "nom", "ville", "email").show()
```

```
RELEVES["nb_clients_sans_commande"] = df_sans_commande.count()

# Observation attendue : 2 clients (Ousmane Sarr, Bineta Mbaye)
```

client_id	nom	ville	email
C007	Ousmane Sarr	Ziguinchor	ousmane.sarr@gmail...
C011	Bineta Mbaye	Mbour	bineta.mbaye@gmail...

À vous de jouer 8.a

Récrivez la détection des **commandes orphelines** (8.1) en **Spark SQL**, sans `left_anti` : un `LEFT JOIN` de `commandes` vers `clients`, en ne gardant que les lignes où `c.client_id IS NULL`.

Indice : `WHERE c.client_id IS NULL` après le `LEFT JOIN`.

```
In [30]: # === À COMPLÉTER ===
spark.sql("""
    SELECT cmd.*
    FROM commandes cmd
    LEFT JOIN clients c ON cmd.client_id = c.client_id
    WHERE c.client_id IS NULL
""").show()
```

commande_id	client_id	produit_id	montant_fcfa	moyen_paiement	statut	date_commande
CMD15	C999	P004	28500	Wave	livree	2025-11-17

9. La fédération totale : ajoutons un CSV à la fête

Le catalogue produits, lui, n'est **dans aucune des deux bases** : l'équipe marketing le tient à jour... dans un fichier CSV (cas très réel). Qu'à cela ne tienne : pour Spark, une source de plus.

9.1 (Re)générer le catalogue `produits.csv`

Comme au TP2, le notebook sait recréer ses données : exécutez, le fichier apparaît à côté du notebook.

```
In [31]: CONTENU_PRODUITS = """produit_id,nom_produit,categorie,prix_fcfa,stock
P001,Smartphone Android,Electronique,145000,8
P002,Casque Bluetooth,Electronique,25000,15
P003,Sac en cuir,Mode,32000,6
P004,Robe wax,Mode,28500,12
P005,Montre connectee,Electronique,57500,0
P006,Chaussures sport,Mode,39000,10
P007,Batterie externe,Electronique,12000,20
P008,Boubou brode,Mode,74500,5
P009,Ecouteurs sans fil,Electronique,15500,18
P010,Ceinture cuir,Mode,8500,9
"""

with open("produits.csv", "w", encoding="utf-8") as f:
    f.write(CONTENU_PRODUITS)

print("Fichier produits.csv écrit (10 produits).")
```

Fichier produits.csv écrit (10 produits).

9.2 Lire le CSV et l'inscrire au catalogue de la session

Notez le retour des réflexes du TP2 (`header` , schéma) : le CSV, lui, n'a pas de SGBD pour nous donner ses types !

```
In [32]: from pyspark.sql.types import (StructType, StructField,
                                         StringType, IntegerType)

schema_produits = StructType([
    StructField("produit_id", StringType(), False),
    StructField("nom_produit", StringType(), False),
    StructField("categorie", StringType(), False),
    StructField("prix_fcfa", IntegerType(), False),
    StructField("stock", IntegerType(), False),
])

df_produits = spark.read.csv("produits.csv", header=True,
                             schema=schema_produits)
df_produits.createOrReplaceTempView("produits")

df_produits.show(3)
print("Vues disponibles :",
      sorted(t.name for t in spark.catalog.listTables()))
```

```
+-----+-----+-----+-----+-----+
|produit_id|      nom_produit| categorie|prix_fcfa|stock|
+-----+-----+-----+-----+-----+
|      P001|Smartphone Android|Electronique|  145000|    8|
|      P002| Casque Bluetooth|Electronique|   25000|   15|
|      P003|      Sac en cuir|      Mode|   32000|    6|
+-----+-----+-----+-----+-----+
only showing top 3 rows
Vues disponibles : ['clients', 'commandes', 'produits']
```

9.3 Trois sources, une requête

`clients` (base MySQL n°1) + `commandes` (base MySQL n°2) + `produits` (fichier CSV) — et une seule requête SQL par-dessus. **C'est ça, la fédération de sources.**

```
In [33]: spark.sql("""
        SELECT c.ville,
               p.categorie,
               SUM(co.montant_fcfa) AS ca_fcfa
        FROM clients AS c
        JOIN commandes AS co ON c.client_id = co.client_id
        JOIN produits AS p ON co.produit_id = p.produit_id
        WHERE co.statut = 'livree'
        GROUP BY c.ville, p.categorie
        ORDER BY ca_fcfa DESC
        """).show()

# Observation attendue : Le duo (Dakar, Electronique) domine.
# Deux bases MySQL + un CSV dans Le meme GROUP BY : mission accomplie !
```

ville	categorie	ca_fcfa
Dakar	Electronique	252500
Dakar	Mode	241500
Rufisque	Electronique	160500
Thies	Electronique	145000
Louga	Mode	74500
Touba	Electronique	57500
Louga	Electronique	12000
Kaolack	Mode	8500
Touba	Mode	8500
Thies	Mode	8500

```
In [34]: # Le CA livre par categorie (commandes x produits -- 2 sources)
df_ca_categorie = spark.sql("""
    SELECT p.categorie,
           COUNT(*) AS nb_ventes,
           SUM(co.montant_fcfa) AS ca_fcfa
    FROM commandes AS co
    JOIN produits AS p ON co.produit_id = p.produit_id
    WHERE co.statut = 'livree'
    GROUP BY p.categorie
    ORDER BY ca_fcfa DESC
    """)
df_ca_categorie.show()

RELEVES["categorie_top_ca"] = df_ca_categorie.first()["categorie"]
```

categorie	nb_ventes	ca_fcfa
Electronique	9	627500
Mode	10	370000

🔧 À vous de jouer 9.a

Le **palmarès des produits** : pour chaque produit ayant au moins une vente **livrée**, affichez `nom_produit`, `categorie`, le nombre de ventes (`nb_ventes`) et le CA (`ca_fcfa`), triés par CA décroissant. Jointure `commandes` × `produits` uniquement.

```
In [35]: spark.sql("""
SELECT
    p.nom_produit,
    p.categorie,
    COUNT(*) AS nb_ventes,
    SUM(cmd.montant_fcfa) AS ca_fcfa
FROM commandes cmd
JOIN produits p ON cmd.produit_id = p.produit_id
WHERE cmd.statut = 'livree'
GROUP BY p.nom_produit, p.categorie
ORDER BY ca_fcfa DESC
""").show()
```

nom_produit	categorie	nb_ventes	ca_fcfa
Smartphone Android	Electronique	3	435000
Boubou brode	Mode	3	223500
Montre connectee	Electronique	2	115000
Sac en cuir	Mode	2	64000
Robe wax	Mode	2	57000
Casque Bluetooth	Electronique	2	50000
Ceinture cuir	Mode	3	25500
Ecouteurs sans fil	Electronique	1	15500
Batterie externe	Electronique	1	12000

10. 🛠 Sous le capot : qui travaille, MySQL ou Spark ?

Jusqu'ici, `dbtable` rapatrie **toute la table** avant que Spark ne filtre. Sur 25 lignes, aucune importance. Sur 500 millions... catastrophe. Deux mécanismes évitent de déplacer des données pour rien :

1. `option("query", ...)` : c'est **MySQL** qui exécute la requête, Spark ne reçoit que le résultat ;
2. le **pushdown de prédicats** : même avec `dbtable`, Spark *pousse* automatiquement les `filter` simples vers MySQL.

10.1 Déléguer une requête à MySQL avec `option("query", ...)`


```
In [36]: df_livrees = (spark.read.format("jdbc")
    .option("url", URL_VENTES)
    .option("query",
        "SELECT commande_id, client_id, montant_fcfa "
        "FROM commandes WHERE statut = 'livree'")
    .option("user", MYSQL_UTILISATEUR)
    .option("password", MYSQL_MDP)
    .option("driver", PILOTE_JDBC)
    .load())

print("Commandes livrees (filtrees PAR MySQL) :", df_livrees.count())
df_livrees.show(3)

# Observation attendue : 19 lignes -- Le WHERE a ete execute dans MySQL,
# seules les lignes utiles ont voyage sur le reseau
```

Commandes livrees (filtrees PAR MySQL) : 19

```
+-----+-----+-----+
|commande_id|client_id|montant_fcfa|
+-----+-----+-----+
|      CMD01|      C001|      145000|
|      CMD02|      C003|      250000|
|      CMD03|      C002|      850000|
+-----+-----+-----+
```

only showing top 3 rows

10.2 La preuve par `explain()` : le pushdown automatique

Même sans `option("query")`, observez le plan d'exécution d'un `filter` sur notre DataFrame JDBC : cherchez la ligne **PushedFilters**.

```
In [37]: df_commandes.filter(F.col("statut") == "livree").explain()

# A Lire dans Le plan : PushedFilters: [*IsNotNull(statut),
# *EqualTo(statut,livree)] -> Spark a pousse le filtre vers MySQL
# tout seul. En Big Data, la regle d'or est : déplacer le calcul
# vers les données, jamais l'inverse.

== Physical Plan ==
*(1) Scan JDBCRelation(commandes) [numPartitions=1] [commande_id#60,client_id#61,
produit_id#62,montant_fcfa#63,moyen_paiement#64,statut#65,date_commande#66] Pushe
dFilters: [*IsNotNull(statut), *EqualTo(statut,livree)], ReadSchema: struct<comm
nde_id:string,client_id:string,produit_id:string,montant_fcfa:int,moyen_paiement:
stri...
```

11. Boucler la boucle : écrire le résultat dans MySQL

Un pipeline ne fait pas que lire : il **livre** ses résultats. Écrivons le CA par ville (section 7.3) dans la base `ecommerce_analytics`, prévue pour ça (et sur laquelle `spark_user` a tous les droits).

⚠ `mode("overwrite")` **remplace** la table à chaque exécution — idéal pour un tableau de bord recalculé, à proscrire pour de l'historisation.

```
In [38]: (df_ca_ville.write.format("jdbc")
          .option("url", URL_ANALYTICS)
          .option("dbtable", "ca_par_ville")
          .option("user", MYSQL_UTILISATEUR)
          .option("password", MYSQL_MDP)
          .option("driver", PILOTE_JDBC)
          .mode("overwrite")
          .save())

print("Table ecommerce_analytics.ca_par_ville ecrite !")
```

Table ecommerce_analytics.ca_par_ville ecrite !

```
In [39]: # La preuve : on relit ce qu'on vient d'ecrire
lire_table_mysql(URL_ANALYTICS, "ca_par_ville").show()

# Verifiez aussi dans Workbench : SELECT * FROM ecommerce_analytics.ca_par_ville
# (cliquez droit sur Les schemas > Refresh ALL pour voir apparaitre la table)
```

```
+-----+-----+-----+
| ville|nb_commandes|ca_fcfa|
+-----+-----+-----+
|  Dakar|          9| 494000|
|Rufisque|          2| 160500|
|   Thies|          2| 153500|
|   Louga|          2|   86500|
|   Touba|          2|   66000|
| Kaolack|          1|    8500|
+-----+-----+-----+
```

12. Le tableau de relevés

La règle du cours : **mesurer, pas affirmer**. Voici ce que vous avez relevé au fil du TP — ces chiffres sont attendus dans votre livrable.

```
In [40]: print(f"{'Mesure':<32} | Valeur")
print("-" * 45)
for cle, valeur in RELEVES.items():
    print(f"{cle:<32} | {valeur}")

# Attendu : 12 clients, 25 commandes, 1 orpheline, 2 clients sans commande,
# CA Livre 969 000 FCFA (ville top : Dakar, categorie top : Electronique)
```

Mesure	Valeur
nb_clients (base CRM)	12
nb_commandes (base ventes)	25
ca_livre_total_fcfa	969000
ville_top_ca	Dakar
nb_commandes_orphelines	1
nb_clients_sans_commande	2
categorie_top_ca	Electronique

13. 🏆 Défis récapitulatifs

À faire en autonomie, en SQL **ou** en API DataFrame. Chaque défi tient en une seule requête / chaîne de transformations.

Défi A — Le taux d'annulation par moyen de paiement

Pour chaque `moyen_paiement` : nombre total de commandes, nombre de commandes annulee, et taux d'annulation en % (arrondi à 1 décimale), trié par taux décroissant.

Indice SQL : `ROUND(100 * SUM(CASE WHEN statut = 'annulee' THEN 1 ELSE 0 END) / COUNT(*), 1)`.

```
In [41]: # === À COMPLÉTER (Défi A) ===
spark.sql("""
    SELECT
        moyen_paiement,
        COUNT(*) AS nb_commandes,
        SUM(CASE WHEN statut = 'annulee' THEN 1 ELSE 0 END) AS nb_annulees,
        ROUND(100 * SUM(CASE WHEN statut = 'annulee' THEN 1 ELSE 0 END) / COUNT(
    FROM commandes
    GROUP BY moyen_paiement
    ORDER BY taux_annulation_pct DESC
    """).show()
```

moyen_paiement	nb_commandes	nb_annulees	taux_annulation_pct
especes	5	2	40.0
Wave	8	1	12.5
carte	4	0	0.0
Orange Money	8	0	0.0

Défi B — La part du mobile money, par catégorie

Pour chaque `categorie` de produit (commandes **livrées** uniquement) : le CA total et la **part du CA payée en mobile money** (Orange Money + Wave), en % arrondi à 1 décimale.

Indice : `CASE WHEN moyen_paiement IN ('Orange Money', 'Wave') THEN montant_fcfa ELSE 0 END`.

```
In [46]: # === À COMPLÉTER (Défi B) ===
spark.sql("""
    SELECT
        p.categorie,
        SUM(co.montant_fcfa) AS ca_total_fcfa,
        ROUND(
            100 * SUM(
                CASE
                    WHEN co.moyen_paiement IN ('Orange Money', 'Wave')
                    THEN co.montant_fcfa
                    ELSE 0
                END
            ) / SUM(co.montant_fcfa),
            1
        ) AS part_mobile_money_pct
    FROM commandes AS co
    JOIN produits AS p ON co.produit_id = p.produit_id
    WHERE co.statut = 'livree'
    GROUP BY p.categorie
    ORDER BY part_mobile_money_pct DESC
""").show(truncate=False)
```

categorie	ca_total_fcfa	part_mobile_money_pct
Electronique	627500	65.8
Mode	370000	55.1

Défi C — L'écriture qui rend service

Écrivez dans `ecommerce_analytics` une table `clients_a_relancer` contenant les clients **sans aucune commande** (section 8.2) avec `client_id`, `nom`, `ville`, `email` — puis relisez-la pour vérifier. L'équipe marketing vous dira merci.

```
In [47]: # === À COMPLÉTER (Défi C) ===
# Écrire les clients à relancer dans la base analytics
(df_sans_commande.select("client_id", "nom", "ville", "email")
 .write.format("jdbc")
 .option("url", URL_ANALYTICS)
 .option("dbtable", "clients_a_relancer")
 .option("user", MYSQL_UTILISATEUR)
 .option("password", MYSQL_MDP)
 .option("driver", PILOTE_JDBC)
 .mode("overwrite")
 .save())

print("Table ecommerce_analytics.clients_a_relancer écrite !")

# Vérification : relire la table
lire_table_mysql(URL_ANALYTICS, "clients_a_relancer").show()
```

Table ecommerce_analytics.clients_a_relancer écrite !

client_id	nom	ville	email
C007	Ousmane Sarr	Ziguinchor	ousmane.sarr@gmail...
C011	Bineta Mbaye	Mbour	bineta.mbaye@gmail...

14. ⚡ Quiz éclair

Répondez de tête, puis dépliez les réponses pour vérifier.

Q1. On code en Python... pourquoi faut-il un fichier `.jar` pour parler à MySQL ?

Q2. Quelle différence entre `option("dbtable", ...)` et `option("query", ...)` ? Laquelle limite les données qui voyagent ?

Q3. Que devient la vue temporaire `clients` quand on arrête la SparkSession ? Et la table MySQL derrière ?

Q4. MySQL sait déjà joindre deux bases d'un même serveur. Citez **trois** apports de Spark que MySQL seul ne fournit pas.

Q5. Qu'affiche `PushedFilters` dans un `explain()`, et pourquoi est-ce crucial en Big Data ?

► 🖱️ Cliquez pour vérifier vos réponses

15. Votre livrable

À pousser sur votre dépôt GitHub **avant la prochaine séance**, dans `notebooks/` :

1. **ce notebook exécuté de bout en bout** (sorties visibles), blocs `À COMPLÉTER` remplis, défis A-B-C traités ;
2. une **capture d'écran Workbench** montrant les tables de `ecommerce_analytics` (preuve de la section 11 et du défi C), dans `docs/` ;
3. le tableau de relevés (section 12) recopié dans votre `README.md` de séance, avec **2 phrases d'analyse métier** (que raconte le CA par ville ? la part mobile money ?).

```
git add notebooks/TP3_pyspark_mysql.ipynb docs/
git commit -m "TP3 : federation MySQL x CSV avec PySpark"
git push
```

⚠️ **Ne poussez jamais** de mot de passe personnel : ici `spark_user` est un utilisateur pédagogique local, mais en entreprise les secrets vont dans des variables d'environnement ou un coffre-fort — jamais dans Git.

✓ Checklist avant de partir

- ☐ `init_tp3.sql` exécuté, les 3 bases visibles dans Workbench
- ☐ notebook exécuté sans erreur, du début à la fin (Kernel ► Restart & Run All)
- ☐ la 25e commande retrouvée (section 8) et expliquée avec vos mots
- ☐ `ecommerce_analytics` contient `ca_par_ville` **et** `clients_a_relancer`
- ☐ livrable poussé sur GitHub, dépôt propre (pas de données, pas de secrets)

La prochaine séance s'appuiera sur ces acquis : mêmes données, plus gros volumes — et les questions d'optimisation commenceront à compter. 🎓